Shopify Pipeline — Optimizations

A record of the performance-relevant changes in this codebase: database-side schema/index/write-pattern work, and the one code-organization change with a real (if modest) performance effect.

1. Database-side optimizations

These come from **migrations/001_rds_perf_pass.sql** and the query/connection patterns in **pipeline/db.py** and **pipeline/summary_incremental.py**. The migration's stated goal was to stop RDS **EBSByteBalance%** (I/O credit balance) from hitting zero on the per-brand MySQL databases.

1.1 Primary keys added to the summary tables

overall_summary, sales_summary, gross_summary, discount_summary, and order_summary each got a primary key on **date**; hour_wise_sales got a composite primary key on **(date, hour)** — applied to both the live tables and their _stage counterparts. Before this, none of these tables had an index on date at all, so every read (the pipeline's own merge logic, and the dashboard) was a full table scan.

1.2 Dropped redundant/duplicate indexes on shopify_orders / shopify_orders_update

shopify_orders went from 18 indexes down to 10: dropped varchar-created_date duplicates of proper indexes, an exact-duplicate composite index, and two typo'd/shadowed product-level composites. shopify_orders_update had three similarly redundant indexes dropped. Every INSERT/UPDATE maintains every index on the table, so this was identified as the single biggest driver of the EBSByteBalance% problem.

1.3 Batched / chunked bulk writes

pipeline/db.py's executemany_chunked runs bulk INSERT ... ON DUPLICATE KEY UPDATE in bounded chunks (EXECUTEMANY_CHUNK_SIZE, default 1000 rows), committing between chunks. One unbounded executemany() over a huge row list becomes one giant transaction holding row locks and growing the redo log for the entire batch — chunking gives MySQL room to flush between commits.

1.4 Staging-table merge instead of per-date DELETE + INSERT

merge_staging_to_main_by_date batches affected dates (5 at a time) into a single range DELETE plus a range INSERT per chunk. Its own docstring documents the before/after: the prior version did a DELETE and an INSERT per individual date in a loop — 2N round-trips and N index-scan deletes for N affected dates.

1.5 Incremental-only summary rebuilds

The entire update_*_incremental family in pipeline/summary_incremental.py recomputes rows only for the dates actually affected by new or changed data on each cycle — never a full-table rebuild on a normal run.

1.6 Connection pooling: NullPool + pool_pre_ping

The SQLAlchemy engines (one per brand, aws_background.py) use NullPool with pool_pre_ping=True — no idle pooled connections held open against RDS between uses, and pool_pre_ping avoids handing out a dead connection after a failover or idle timeout.

1.7 Concurrency cap on per-brand processing

MAX_CONCURRENT_BRANDS (default 3, pipeline/orchestration.py) deliberately limits how many brands' ETL passes — each a heavy set of multi-table JOINs and summary rebuilds — run at once, specifically to protect shared RDS I/O rather than firing every brand in parallel.

1.8 Connection retry and SSL-fallback

get_db_connection retries with exponential backoff plus jitter, and automatically falls back to a non-SSL connection on an SSL-specific failure. This is a reliability measure more than a throughput one, but it avoids reconnect storms under transient failure.

2. Code-organization optimization

2.1 Dead code removal (pre-modularization step)

Before aws_background.py was split into the pipeline/ package, a first pass (commit 19c4fca) removed a fully-shadowed duplicate block of the checkout-initiated/buy-now Mongo sync functions — an entire first definition that was completely overwritten by later redefinitions of the same names and never executed — plus roughly ten other confirmed-unused functions and the now-unused smtplib / email.mime / multiprocessing imports.

Note: the module split itself (aws_background.py → pipeline/*.py) was explicitly NOT a performance change. Every step moved code verbatim behind a facade re-export, with object-identity checks confirming shared singletons (DB engine pool, HTTP session, scheduler, Flask app) stayed the same objects across modules rather than being duplicated. It's a maintainability refactor with a zero-behavior-change guarantee, not an optimization pass.

3. Open finding — verify before relying on it

migrations/001_rds_perf_pass.sql also claims the pipeline was "patched to use created_dt / created_hr (generated, indexed columns) instead of VARCHAR created_date + HOUR(created_time)", and drops the old varchar-based indexes on that assumption. This does **not** match the current pipeline/ package: pipeline/summary_incremental.py still queries via STR_TO_DATE(created_date, '%Y-%m-%d') and HOUR(created_time) everywhere, including the payment-mode-breakdown queries added most recently. The created_dt/created_hr pattern only exists in backfill scripts/aws_data_update.py, a separate standalone script — not the live pipeline. If Phase 2/2b's index drops from that migration were ever actually run against a brand's production database, the pipeline's real queries may now be missing index support they used to have. Worth an EXPLAIN check on the live hour_wise_sales / order_summary queries against a production brand database.